

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»**

Отчёт по лабораторной работе
«Лабораторная работа №2 по ЧМА»

Выполнили работу:
Косарев И. А., J3110
Кащеев М. Н., J3111
Капустина Ю. И., J3110

Санкт-Петербург
2025

Оглавление

Введение	2
Теоретическая часть	3
2.1 Степенной метод	3
2.2 Сингулярное разложение	3
2.3 Метод Якоби	4
Практическая часть	6
3.1 Особые моменты	6
3.2 Код реализации	6
3.3 Анализ и построение графиков	9
Выводы	11

Введение

Цель: познакомиться с понятиями сингулярного разложения, степенного метода и метода Якоби; применить итерационные методы на практике.

Задачи

- Изучить понятия сингулярного разложения
- Реализовать алгоритм степенного метода
- Реализовать алгоритм сингулярного разложения
- Реализовать алгоритм плоских вращений Якоби
- Обработать изображение, используя указанные методы

Теоретическая часть

2.1 Степенной метод

Пусть A — квадратная матрица размера $n \times n$. Степенной метод позволяет найти наибольшее по модулю собственное значение λ_1 и соответствующий собственный вектор $\mathbf{v}_1$. Алгоритм нахождения собственного вектора:

1. Выбираем начальный вектор : $\mathbf{v}_0$ - не нулевой.
2. Умножаем матрицу на текущий вектор:

$$\mathbf{v}_k = A\mathbf{v}_{k-1}$$

3. Нормируем полученный вектор:

$$\mathbf{v}_k = \frac{\mathbf{v}_k}{\|\mathbf{v}_k\|}$$

4. Проверяем условие сходимости:

$$\|\mathbf{v}_k - \mathbf{v}_{k-1}\| < \varepsilon$$

Если условие выполняется, процесс завершается. Если нет - продолжаем выполнять пункты 2 и 3.

5. Вычисляем оценку Рэлея для собственного значения:

$$\lambda = \frac{\mathbf{v}^T A \mathbf{v}}{\mathbf{v}^T \mathbf{v}}$$

Метод возвращает приближенное значение наибольшего собственного значения λ и соответствующий собственный вектор $\mathbf{v}$.

2.2 Сингулярное разложение

Для любой матрицы A размера $m \times n$ существует сингулярное разложение (SVD):

$$A = U\Sigma V^T$$

где:

- U - матрица левых сингулярных векторов (столбцы $\mathbf{u}_i$),
- Σ - диагональная матрица сингулярных чисел ($\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$),

- V - матрица правых сингулярных векторов (столбцы $\mathbf{v}_i$).

Связь с собственными векторами:

- Правые сингулярные векторы $\mathbf{v}_i$ - собственные векторы матрицы $A^T A$
- Левые сингулярные векторы $\mathbf{u}_i$ - собственные векторы матрицы AA^T

Алгоритм нахождения сингулярных векторов

- Шаг 1: Вычисление $A^T A$ и AA^T

$$A^T A \quad (\text{матрица } n \times n) \quad \text{и} \quad AA^T \quad (\text{матрица } m \times m)$$

- Шаг 2: Нахождение правых сингулярных векторов $\mathbf{v}_i$

1. Решаем уравнение:

$$(A^T A)\mathbf{v}_i = \lambda_i \mathbf{v}_i$$

2. Нормируем $\mathbf{v}_i$ (чтобы $\|\mathbf{v}_i\| = 1$)

3. Столбцы V - это $\mathbf{v}_i$, упорядоченные по убыванию λ_i

- Шаг 3: Нахождение левых сингулярных векторов $\mathbf{u}_i$

1. Решаем уравнение:

$$(AA^T)\mathbf{u}_i = \lambda_i \mathbf{u}_i$$

2. Нормируем $\mathbf{u}_i$

3. Столбцы U - это $\mathbf{u}_i$, упорядоченные по убыванию λ_i

- Шаг 4: Связь между $\mathbf{u}_i$ и $\mathbf{v}_i$ Если $\sigma_i \neq 0$, то:

$$\mathbf{u}_i = \frac{A\mathbf{v}_i}{\sigma_i}, \quad \mathbf{v}_i = \frac{A^T \mathbf{u}_i}{\sigma_i}$$

- Шаг 5: Построение Σ

$$\Sigma = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r), \quad \sigma_i = \sqrt{\lambda_i}$$

2.3 Метод Якоби

Метод Якоби является итерационным алгоритмом, который предназначен для диагонализации симметричных матриц путём последовательного обнуления их элементов вне диагонали с помощью ортогональных преобразований (то есть вращений). В результате матрица становится диагональной, а её элементы на диагонали становятся собственными значениями.

Для симметрической матрицы $A \in \mathbb{R}^{n \times n}$ всегда существует ортогональная матрица Q , такая, что:

$$Q^T A Q = D,$$

где D — диагональная матрица собственных значений λ_i .

Плоское вращение

Плоское вращение происходит при построении матрицы Якоби: на каждой итерации выбирается пара индексов (p, q) и строится матрица вращения J , которая обнуляет элемент A_{pq} после преобразования:

$$A' = J^T A J.$$

Матрица J имеет вид:

$$J = \begin{pmatrix} 1 & \cdots & 0 & \cdots & 0 \\ \vdots & \cos \theta & \cdots & -\sin \theta & \vdots \\ 0 & \cdots & 1 & \cdots & 0 \\ \vdots & \sin \theta & \cdots & \cos \theta & \vdots \\ 0 & \cdots & 0 & \cdots & 1 \end{pmatrix},$$

где θ — угол вращения, вычисляемый из условия обнуления A_{pq} .

Вычисление угла θ

Угол θ определяется формулой:

$$\theta = \frac{1}{2} \arctan \left(\frac{2a_{pq}}{a_{pp} - a_{qq}} \right).$$

Данная формула получается из условия обнуления элемента a'_{pq} после преобразования $A' = J^T A J$. Элемент a'_{pq} равен:

$$A'_{pq} = \frac{1}{2}(A_{pp} - A_{qq}) \sin(2\theta) + A_{pq} \cos(2\theta).$$

Приравнявая $a'_{pq} = 0$, получаем:

$$\tan(2\theta) = \frac{2a_{pq}}{a_{qq} - a_{pp}}.$$

Критерий остановки

Критерий остановки: норма внедиагональных элементов $\|\text{off}(A)\|_F$ (сумма квадратов внедиагональных элементов) становится меньше заданного допуска ϵ . Также возможно взятие максимального модуля внедиагонального элемента и сравнение его с заданным значением ϵ в качестве критерия остановки.

Доказательство сходимости

После каждого вращения сумма квадратов внедиагональных элементов уменьшается на $2A_{pq}^2$. Поскольку на каждой итерации выбирается максимальный внедиагональный элемент, последовательность $\|\text{off}(A)\|_F$ (то есть фробениусова норма, которая вычисляется как: $\sqrt{\sum_{i \neq j} A_{ij}^2}$) монотонно убывает и стремится к нулю.

Практическая часть

3.1 Особые моменты

В процессе выполнения лабораторной работы было замечено, что в процессе вычисления собственных векторов матрицы ковариации могли возникать отрицательные собственные значения. Вероятно, это происходило из-за погрешности вычислений.

Код реализованных методов и их применение представлены в репозитории лабораторной работы в папке *lab2*: [репозиторий лабораторной](#).

3.2 Код реализации

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

def power_method(
    cov: np.array, max_iter: int = 50, tol: float = 1e-8
) -> tuple[float, np.array]:
    """Степенной метод для нахождения собственного значения максимального по модулю
    и соответствующего ему собственного вектора симметрической матрицы

    Args:
        cov (np.array): матрица (ковариации)
        max_iter (int, optional): кол-во итераций алгоритма. Defaults to 50.
        tol (float, optional): пороговая погрешность

    Raises:
        ValueError: в случае невалидного max_iter
        ValueError: в случае, если передаваемый тензор - не матрица
        ValueError: если матрица несимметрическая

    Returns:
        tuple[float, np.array]: собственное значение максимального модуля и соот. ему вектор
    """
    if max_iter <= 0:
        raise ValueError("Invalid max_iter value")
    if cov.ndim != 2:
        raise ValueError("2-dimensional matrices allowed only")
    if cov.shape[0] != cov.shape[1]:
        raise ValueError("Square matrices allowed only")

    n = cov.shape[0]
```

```

eigenvec = np.random.uniform(-1, 1, n)
eigenvec /= np.linalg.norm(eigenvec)
prev = eigenvec
eigenvec = cov @ eigenvec

for _ in range(max_iter - 1):
    if np.linalg.norm(eigenvec - prev) < tol:
        break
    prev = eigenvec
    eigenvec = cov @ eigenvec
    eigenvec /= np.linalg.norm(eigenvec)

lam = (eigenvec.T @ cov @ eigenvec) / (eigenvec.T @ eigenvec)

return (lam, eigenvec)

def svd_via_power_method(
    X: np.array,
    max_iter: int = 50,
    enable_below_zero_sigma_warning: bool = True,
) -> tuple[np.array, np.array, np.array]:
    """Сингулярное разложение на основе степенного метода

    Args:
        X (np.array): матрица данных
        max_iter (int, optional): кол-во итераций. Defaults to 50.
        enable_below_zero_sigma_warning (bool, optional): предупреждать ли
        об отрицательных значениях. Defaults to True.

    Raises:
        ValueError: в случае невалидного max_iter
        ValueError: если передаваемый тензор - не матрица

    Returns:
        tuple[np.array, np.array, np.array]: U, S, V^T, где:
        U - матрица левых сингулярных векторов,
        S - массив сингулярных чисел,
        V - матрица правых сингулярных векторов
    """
    if max_iter <= 0:
        raise ValueError("Invalid max_iter value")
    if X.ndim != 2:
        raise ValueError("2-dimensional matrices allowed only")

    cov = X.T @ X

    U = []
    sigmas = []
    V = []

    rank = np.linalg.matrix_rank(X)

    for k in range(rank):

```

```

lam, v = power_method(cov, max_iter)
v = v.reshape(X.shape[1], 1)

if lam < 0:
    if enable_below_zero_sigma_warning:
        print(
            f"Below zero singular value detected at iteration {k}: {lam:.3e}"
        )
    sigma = np.sqrt(-lam)
else:
    sigma = np.sqrt(lam)
sigmas.append(sigma)
V.append(v.T)
U.append(X @ v / sigma)

cov = cov - (v @ v.T) * lam
return (
    np.concatenate(U, axis=1),
    np.array(sigmas),
    np.concatenate(V, axis=0),
)

def trunc_recon(U, S, Vt, r=300) -> np.array:
    r = min([r, U.shape[1], S.shape[0], Vt.shape[0]])
    return U[:, :r] @ np.diag(S)[:r, :r] @ Vt[:, :r]

def jacobi_eigenvalue(
    A: np.array, tol: float = 1e-6, max_iter: int = 1000
) -> tuple[np.array, np.array, np.array]:
    """Найти собственные векторы и значения симметрической матрицы симметрической матрицы
    методом плоских вращений Якоби

    Args:
        A (np.array): симметрическая матрица
        tol (float, optional): порог ошибки нуля, после которого
        алгоритм завершается. Defaults to 1e-6.
        max_iter (int, optional): максимальное кол-во операций. Defaults to 1000.

    Raises:
        ValueError: в случае невалидного max_iter
        ValueError: в случае, если переданный тензор - не матрица
        ValueError: если переданная матрица неквадратная

    Returns:
        tuple[np.array, np.array, np.array]: _description_
    """
    if max_iter <= 0:
        raise ValueError("Invalid max_iter value")
    if len(A.shape) != 2:
        raise ValueError("Only matrices allowed!")
    if A.shape[0] != A.shape[1]:
        raise ValueError("Only square matrices allowed!")

```

```

n = A.shape[0]
D = A.copy()

error_history = []
err = np.linalg.norm(np.triu(A, 1))
error_history.append(err)

eigenvecs = np.eye(n)

for i in range(max_iter):

    p, q = np.unravel_index(
        np.argmax(np.abs(D - np.diag(np.diag(D)))), A.shape
    )
    if abs(D[p, q]) < tol:
        break

    # Вычисляем угол вращения
    if D[p, p] == D[q, q]:
        theta = np.pi / 4
    else:
        theta = 0.5 * np.arctan(2 * D[p, q] / (D[p, p] - D[q, q]))

    # Формируем матрицу вращения
    J = np.eye(n)
    c = np.cos(theta)
    s = np.sin(theta)
    J[p, p] = c
    J[q, q] = c
    J[p, q] = -s
    J[q, p] = s

    # Применяем вращение к матрице D
    D = J.T @ D @ J
    err = np.linalg.norm(np.triu(D, 1))
    error_history.append(err)

    eigenvecs = eigenvecs @ J

D = np.diag(D)

return np.sort(D)[::-1], eigenvecs, error_history

```

3.3 Анализ и построение графиков

В процессе выполнения лабораторной были построены различные графики (представлены в *репозитории лабораторной*). Основные выводы:

- С увеличением кол-ва итераций в степенном методе "разброс" экспериментальных сингулярных значений от их истинного уменьшается.
- С увеличением ранга приближения по сингулярному разложению "картинка" каждого из цветовых каналов становится четче.

- С достаточно большим кол-вом итераций разница между собственными значениями, найденными методами Якоби и степенным, уменьшается, однако для полного соответствия методу Якоби требуется значительно больше итераций.

В результате после выполнения приближения по каждому из каналов была получена следующая картинка:

Восстановленное RGB изображение (приближенная SVD, $r = 25$)

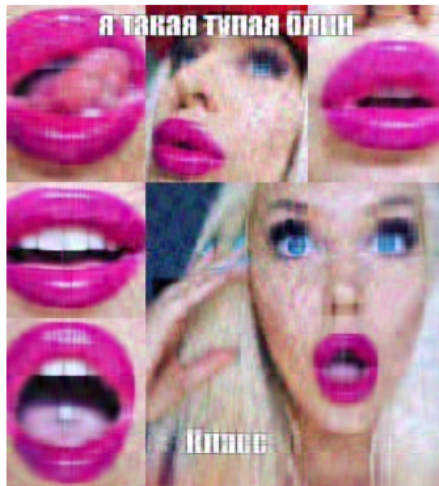


Рис. 3.1: Полученная картинка

Выводы

В результате выполнения лабораторной работы мы познакомились с понятиями сингулярного разложения, степенного метода и метода плоских вращений Якоби на практике. Мы выявили закономерности между точностью вычисления собственных/сингулярных значений и векторов и кол-ва итераций разных методов, выполнили сегментацию изображения по каналам и совершили усечение разложения, применив полученные теоретические знания на практике.